

NSS App: Comprehensive Manual Test Plan

This document outlines the step-by-step test sequences required to validate all permutations of the attendance and penalty logic in the NSS IITP Official App.

Phase 1: Standard Event Workflows (Normal Events)

Test 1.1: Normal Event - QR Code & Manual Override

1. **Create Event:** Type "Normal", Mode: QR Code, Positive Hours: 2.0, Mandatory: OFF.
2. **Start Attendance:** Tap "Start Attendance". Wait for Live Count to appear.
3. **Add Manual:** Tap "Add/Delete Roll No". Add a test student (e.g., 2301CT01).
4. **Close Event:** Tap "Close Event".
5. **Verify:** Student 2301CT01 should have +2.0 hours.

Test 1.2: Normal Event - Geo-Tagged Photo Approval

1. **Create Event:** Type "Normal", Mode: Geo-Tagged Photo, Positive Hours: 3.0, Mandatory: OFF.
2. **Student Action:** Student submits a photo.
3. **Admin Action:** Go to "Pending Photos". Approve the photo.
4. **Verify:** Student receives +3.0 hours immediately.

Phase 2: Mandatory Event Workflows & Penalties

Test 2.1: Mandatory Event - Applying Penalty to Absentees

1. **Create Event:** Mode: Both (QR & Geo), Positive Hours: 2.0, Mandatory: ON, Negative Hours: 1.5.
2. **Add Manual:** Add Student A manually (+2.0 hours).
3. **Close Event:** Close the event. Student B is completely absent.
4. **Apply Penalty:** Tap "Apply Penalty".
5. **Verify:** Student B should lose 1.5 hours. Student A's hours remain unchanged.

Test 2.2: Mandatory Event - Selective Penalty Exemptions (Zero Roll Numbers)

1. **Create Event:** Mode: QR Code, Positive Hours: 2.0, Mandatory: ON, Negative Hours: 2.0.
2. **Close Event & Apply Penalty:** Student C is absent. During the "Apply Penalty" screen, manually move Student C to the "Exempt" list (Zero Roll Numbers).
3. **Verify:** Student C should receive 0 penalty hours, not -2.0.

Phase 3: Edge Cases & Corrections

Test 3.1: Editing Hours of an Active Event

1. **Create Event:** Positive Hours: 2.0.
2. **Add Manual:** Add Student D (+2.0 hours).
3. **Edit Event:** Long press the event card. Change Positive Hours to 4.0.
4. **Verify:** Student D's total hours should automatically jump from +2.0 to +4.0.

Test 3.2: Deleting Attendance

1. **Pre-requisite:** Student E has +3.0 hours from an event.
2. **Delete Attendance:** Long press "Add/Delete Roll No". Enter Student E's roll number to mark them absent.

3. **Verify:** Student E's hours should drop by 3.0.

Test 3.3: Reopening a Penalized Event (Make Live) & Refunding

1. **Setup:** Mandatory event closed, penalty applied. Student F was absent and got penalized (-2.0).
2. **Make Live:** Go to Event History, tap "Make Live".
3. **Late Attendance:** Add Student F manually via "Add/Delete Roll No".
4. **Expected Outcome:** Student F receives the +2.0 positive hours AND their -2.0 penalty is successfully refunded.

⚠ Phase 4: Known Issues (Do Not Test on Live Data)

Based on a deep-dive code audit, the following critical issues exist in the backend and may crash your app if triggered during manual testing.

Issue 1: The "Apply Penalty" Batch Limit Crash (CRITICAL)

Description: When applying a penalty to a mandatory event, the backend shoves all absent students into a single Firestore `batch.commit()`. Firestore hard-limits batches to 500 documents.

How to trigger it: Create a mandatory event. Close it with 0 attendees. Click "Apply Penalty". If there are more than 500 non-admin users in your `users` collection, the backend will attempt >500 writes and instantly crash. The penalty will fail to apply.`

Current Status: You have 419 users (383 students) in production. You are safe right now, but the app will permanently break as soon as your total student count exceeds ~500 (e.g., during your next fresher intake).

Issue 2: "Apply Penalty" Concurrency Race Condition

Description: The `applyAbsentPenalty` cloud function does not use a database transaction. It uses a sequence of reads followed by a delayed batch write.

How to trigger it: If two admins click the "Apply Penalty" button at the exact same time (or if one admin double-clicks due to network lag), both requests read the database before the lock is applied. Both requests will execute, deducting the penalty from students TWICE (e.g., -4.0 hours instead of -2.0).